

Video creator

IMPORTANT!

This program requires the following dependencies to run properly:

- ExifTool: For extracting GPS and date metadata from media files
- FFmpeg: For video processing, segment creation, and audio handling
- OpenAI API Key: Required for image generation, script creation, and text-to-speech

Setup Instructions:

1. Set the environment variable 'OpenAIToken' with your API key
2. Ensure ExifTool and FFmpeg are installed and in your system PATH
3. Prepare your media files (images and/or videos) with GPS and date metadata
4. Run the Main class and follow a few steps shown in console
5. Enjoy!

General Information:

This program creates AI-powered videos using the provided files by the user.

The workflow includes:

1. Extracting metadata (GPS coordinates and dates) from each file
2. Sorting files chronologically from oldest to newest
3. Generating two AI images: an artistic 'essence' image and a geographic map
4. Creating a narration script based on visual analysis of the media
5. Converting the script to speech audio
6. Compiling all elements into a portrait-format video with some cool visual effects effects and audio specifications

The final output is a video optimized for platforms with portrait formats like YouTube, Instagram, and TikTok, with audio normalized to YouTube standards (-14 LUFS, -1 dBTP, 7 LRA).

CLASS - Main

Purpose: Entry point of the program. Guides the entire workflow from media input to final video output.

Attributes:

TYPE	DESCRIPTION
sc	Scanner to read user input from console

Methods:

PARAMETERS	RETURN	DESCRIPTION
main(String[] args)	void	Coordinates the entire video creation process: requests media file paths, extracts metadata, sorts files by date, generates AI images and script, creates audio narration, compiles video segments, and converts to portrait format with optional audio verification

Workflow:

1. Validates OpenAI API key
2. Asks for number of media files and their paths
3. Sorts files chronologically
4. Generates “essence” image
5. Generates geographic map showing oldest and newest locations
6. Creates narration script from visual analysis
7. Converts script to audio
8. Creates video segments (1920x1080)
9. Compiles segments with audio (delayed by 3 seconds)
10. Converts to portrait format (1080x1920 & adds blur effect)
11. Optionally → verifies audio loudness levels

CLASS - mediaFile

Purpose: Represents a single media file with its path and extracted metadata (GPS coordinates and date).

Attributes:

TYPE	DESCRIPTION
mediaPath	String containing the file path
GPS	String containing GPS coordinates extracted from metadata
date	String containing creation date in format 'yyyy:MM:dd HH:mm:ss'

Constructor:

PARAMETERS	DESCRIPTION
mediaFile(String PATH)	Constructs a mediaFile object, stores the path, and immediately calls getData() to extract metadata

Methods:

PARAMETERS	RETURN	DESCRIPTION
getData(String PATH)	void	Private method that uses metaExtract to extract GPS and date metadata from the file

CLASS - metaExtract

Purpose: Extracts metadata from media files using ExifTool, specifically GPS coordinates and creation date.

Methods:

PARAMETERS	RETURN	DESCRIPTION
metaExtractGPS(String path)	String	Extracts GPS coordinates using exiftool -GPSPosition. Returns 'No data' if not found
metaExtractDate(String path)	String	Extracts creation date using exiftool -CreateDate. Returns 'No data' if not found
runExiftool(String path, String tag)	String	Private method that executes ExifTool with specified tag and returns the extracted value. Uses -s3 flag for clean output

CLASS - mediaUtils

Purpose: Utility class for sorting and analyzing mediaFile arrays based on dates.

Methods:

PARAMETERS	RETURN	DESCRIPTION
parseDate(String dateStr)	LocalDateTime	Private method that parses date string in format 'yyyy:MM:dd HH:mm:ss'. Returns LocalDateTime.MIN on parse failure
sortByDate(mediaFile[] media)	void	Sorts the mediaFile array chronologically from oldest to newest using parseDate()
getOldest(mediaFile[] media)	mediaFile	Returns the first element (oldest) from the sorted array
getNewest(mediaFile[] media)	mediaFile	Returns the last element (newest) from the sorted array

CLASS - apiHandler

Purpose: Handles all interactions with the OpenAI API, including image generation, text generation, audio synthesis, and multimodal analysis.

Methods:

METHOD	PARAMETERS	RETURN	DESCRIPTION
buildImagePrompt	mediaFile[] media	String	Creates prompt for the 'essence' image that artistically represents all media files
buildMapPrompt	mediaFile oldest, mediaFile newest	String	Creates prompt for geographic map showing oldest and newest locations with motivational quote
buildScriptPrompt	mediaFile[] media	String	Creates prompt for narration script focusing on visual content analysis
generateImage	String prompt, String apiKey, String outputName	void	Generates image using gpt-image-1 model (1536x1024) and saves as PNG file
generateText	String prompt, String apiKey	String	Generates text using gpt-4o-mini model via chat completions endpoint
generateAudio	String text, String apiKey, String outputName	void	Converts text to speech using gpt-4o-mini-tts with 'alloy' voice and saves as MP3
generateTextFromMedia	mediaFile[] media, String apiKey	String	Performs multimodal analysis: extracts frames from videos, encodes images as base64, sends to API for visual analysis and script generation

Private Methods:

METHOD	DESCRIPTION
buildMultimodalChatJson	Constructs JSON payload with text and image data for multimodal API requests
extractRepresentativeFrame	Uses FFmpeg to extract frame at 00:00:01 from videos for visual analysis
fileToDataURL	Converts image files to base64 data URLs for API transmission
contentTypeFor	Determines MIME type from file extension (supports png, jpg, webp, gif, bmp, heic, tiff)
isVideo	Checks if file is video based on extension (mp4, mov, avi, mkv, webm, m4v)

CLASS - videoCreator

Purpose: Handles all video processing operations using FFmpeg: segment normalization, video compilation, portrait conversion, and audio loudness verification.

Methods:

METHOD	PARAMETERS	RETURN	DESCRIPTION
createVideo	mediaFile[] media, String audioPath	void	Creates normalized video segments (1920x1080, 30fps, H.264), concatenates them with audio (delayed 3 seconds), and outputs final_video.mp4
createImageSegment	File imageFile, int durationSeconds, String outputName	File	Private method that converts still image to horizontal video segment (1920x1080) with specified duration
createVideoSegment	File videoFile, String outputName	File	Private method that normalizes video to horizontal format (1920x1080, 30fps) and removes audio
convertToPortrait	String inputVideo, String outputVideo	void	Converts horizontal video to portrait (1080x1920) with blurred background using complex filter chains
verifyAudioLevels	String videoPath	void	Analyzes and displays audio loudness metrics (LUFS, dBTP, LRA) compared to YouTube standards
isVideo	String path	boolean	Private method that determines if file is video based on extension
runFFmpeg	String audioPath	void	Private method that concatenates video

			segments with audio, applying loudness normalization (-14 LUFS, -1 dBTP, 7 LRA)
runCommand	String[] command, String label	int	Private method that executes FFmpeg commands and captures exit codes

Video Processing Details:

Segment Normalization:

- Resolution: 1920x1080 (horizontal)
- Frame rate: 30 fps
- Codec: libx264
- Pixel format: yuv420p
- Still images: 3-second duration each

Audio Normalization (YouTube Standards):

- Integrated Loudness: -14 LUFS
- True Peak: -1 dBTP
- Loudness Range: 7 LU
- Audio Bitrate: 192 kbps (AAC)
- Audio Delay: 3 seconds (starts after opening image)

Portrait Conversion:

- Output resolution: 1080x1920 (portrait)
- Background: Blurred and stretched version of source
- Foreground: Letterboxed original content centered
- Blur settings: boxblur=20:10

Relationships Between Classes:

- Main instantiates and coordinates all other classes
- Main creates mediaFile objects for each user-provided file
- mediaFile uses metaExtract to populate GPS and date attributes
- mediaUtils processes arrays of mediaFile objects for sorting and analysis
- apiHandler receives mediaFile arrays and generates AI content
- videoCreator receives mediaFile arrays and audio path to produce final video
- All classes use the file system for intermediate file storage

File Flow and Dependencies:

Input Files:

- User-provided media files (images: jpg, png, webp, gif, bmp, heic, tiff; videos: mp4, mov, avi, mkv, webm, m4v)

Generated Files:

1. essenceImage.png - AI-generated artistic representation
2. mapImage.png - AI-generated geographic map
3. narration.mp3 - Text-to-speech audio narration
4. segment_000.mp4 - Opening essence image (3 seconds)
5. segment_1.mp4, segment_2.mp4, ... - User media segments
6. segment_last.mp4 - Closing map image (3 seconds)
7. input.txt - FFmpeg concat file listing all segments
8. final_video.mp4 - Horizontal compiled video
9. [user-specified-name].mp4 - Final portrait video with blur effects

Error Handling and Limitations:

Known Limitations:

- HEIC/HEIF images may fail if FFmpeg lacks the decoder
- RAW image formats (CR2, NEF, ARW, DNG, etc.) are not supported
- Files without GPS or date metadata will return 'No data'
- Video files without metadata use fallback date (LocalDateTime.MIN)
- Original video audio is removed and replaced with generated narration
- Video analysis uses only the frame at 00:00:01 (not full video content)

Error Recovery:

- Invalid API key causes immediate program termination
- FFmpeg failures throw RuntimeException with exit code
- Missing images (essenceImage.png, mapImage.png) trigger warnings but continue processing
- ExifTool errors return 'Error: [message]' instead of crashing

External Dependencies:

TOOL	PURPOSE	USAGE
ExifTool	Metadata extraction	Extracts GPS coordinates and creation dates from images and videos
FFmpeg	Video/audio processing	Frame extraction, video normalization, segment concatenation, audio mixing, format conversion, loudness normalization
OpenAI API	AI content generation	Image generation (gpt-image-1), text generation (gpt-4o-mini), multimodal analysis, text-to-speech (gpt-4o-mini-tts)
curl	HTTP requests	Communicates with OpenAI API endpoints for all AI operations

Performance Considerations:

- Processing time scales with number of files (3-10 minutes typical for 5-10 files)
- Large video files significantly increase processing time
- API calls introduce network latency (each image generation: 10-30 seconds)
- FFmpeg operations are CPU-intensive (video encoding, filtering)
- Temporary files are created and deleted automatically
- Portrait conversion with blur is the most resource-intensive step

For best results:

- Use media files with GPS and date metadata
- Provide 3-12 files for optimal narrative flow
- Check that ExifTool and FFmpeg are properly installed
- Have a valid OpenAI API key with sufficient credits